

## **EXPERIMENT-7**

### **SIDDHI SAMBHAV**

### **(24UADS1070)**

**Objective :-** WAP to retrain a pretrained imagenet model to classify a medical image dataset.

**Description :-** This project is based on applying deep learning techniques for medical image classification using transfer learning. Instead of training a neural network from scratch, we used a pre-trained model like ResNet18 or MobileNetV2, which is already trained on the ImageNet dataset. This helps in saving time and improving accuracy because the model already knows how to extract useful features from images.

In this experiment, we worked on a medical dataset containing images such as chest X-rays (Normal and Pneumonia cases). The dataset was already divided into training and validation sets, which made the process easier.

Before training, all images were preprocessed by resizing them to a fixed size and converting them into tensors. Some transformations like normalization and flipping were applied to make the model more robust.

The pre-trained model was then modified by replacing its final layer so that it can classify our specific medical categories. After that, the model was trained using the training dataset and evaluated on the validation dataset.

During training, we observed the accuracy of the model for both training and validation data. The results were visualized using graphs, and performance was further analyzed using confusion matrix and classification report.

Overall, this project demonstrates how transfer learning can be effectively used for solving real-world problems like medical image classification with high accuracy and less training time.

## Source Code :-

```
import os
import copy
import time
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim import lr_scheduler
from torch.utils.data import DataLoader
from torchvision import datasets, models, transforms

from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

# =====
# CONFIG
# =====
DATA_DIR = "chest_xray"
BATCH_SIZE = 8
NUM_EPOCHS = 10
NUM_CLASSES = 2
DEVICE = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

# =====
# TRANSFORMS
# =====
data_transforms = {
    'train': transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406],
                              [0.229, 0.224, 0.225])
    ]),
    'val': transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406],
                              [0.229, 0.224, 0.225])
    ]),
}

# =====
# LOAD DATA
```

```

# =====
image_datasets = {
    x: datasets.ImageFolder(os.path.join(DATA_DIR, x), data_transforms[x])
    for x in ['train', 'val']
}

dataloaders = {
    x: DataLoader(image_datasets[x], batch_size=BATCH_SIZE, shuffle=True)
    for x in ['train', 'val']
}

dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val']}
class_names = image_datasets['train'].classes

# =====
# MODEL
# =====
model = models.resnet18(pretrained=True)

for param in model.parameters():
    param.requires_grad = True

num_fts = model.fc.in_features
model.fc = nn.Linear(num_fts, NUM_CLASSES)

model = model.to(DEVICE)

# =====
# LOSS + OPTIMIZER
# =====
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.0003)
scheduler = lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)

# =====
# TRAIN FUNCTION
# =====
def train_model(model, num_epochs):
    best_acc = 0.0
    best_model_wts = copy.deepcopy(model.state_dict())

    train_acc_history = []
    val_acc_history = []

    for epoch in range(num_epochs):
        print(f"Epoch {epoch+1}/{num_epochs}")

```

```

print("-" * 10)

for phase in ['train', 'val']:
    if phase == 'train':
        model.train()
    else:
        model.eval()

    running_corrects = 0

    for inputs, labels in dataloaders[phase]:
        inputs = inputs.to(DEVICE)
        labels = labels.to(DEVICE)

        optimizer.zero_grad()

        with torch.set_grad_enabled(phase == 'train'):
            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)
            loss = criterion(outputs, labels)

            if phase == 'train':
                loss.backward()
                optimizer.step()

            running_corrects += torch.sum(preds == labels.data)

    if phase == 'train':
        scheduler.step()

    epoch_acc = running_corrects.double() / dataset_sizes[phase]
    print(f"{phase} Acc: {epoch_acc:.4f}")

    if phase == 'val':
        val_acc_history.append(epoch_acc.cpu().numpy())
        if epoch_acc > best_acc:
            best_acc = epoch_acc
            best_model_wts = copy.deepcopy(model.state_dict())
    else:
        train_acc_history.append(epoch_acc.cpu().numpy())

print()

model.load_state_dict(best_model_wts)
return model, train_acc_history, val_acc_history

```

```

# =====
# TRAIN
# =====
model, train_acc, val_acc = train_model(model, NUM_EPOCHS)

# =====
# PLOT
# =====
plt.plot(train_acc, label="Train")
plt.plot(val_acc, label="Val")
plt.legend()
plt.title("Accuracy")
plt.savefig("accuracy.png")
plt.show()

# =====
# CONFUSION MATRIX
# =====
all_preds = []
all_labels = []

model.eval()
with torch.no_grad():
    for inputs, labels in dataloaders['val']:
        inputs = inputs.to(DEVICE)
        labels = labels.to(DEVICE)

        outputs = model(inputs)
        _, preds = torch.max(outputs, 1)

        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

cm = confusion_matrix(all_labels, all_preds)

sns.heatmap(cm, annot=True)
plt.title("Confusion Matrix")
plt.savefig("cm.png")
plt.show()

print(classification_report(all_labels, all_preds))

# =====
# SAVE MODEL
# =====
torch.save(model.state_dict(), "model.pth")

```

```
print("Model saved!")
```

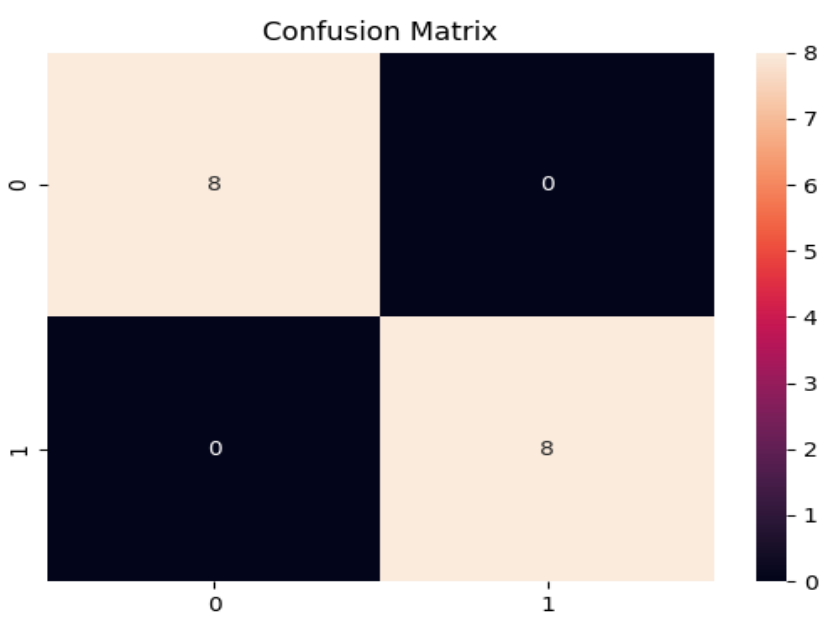
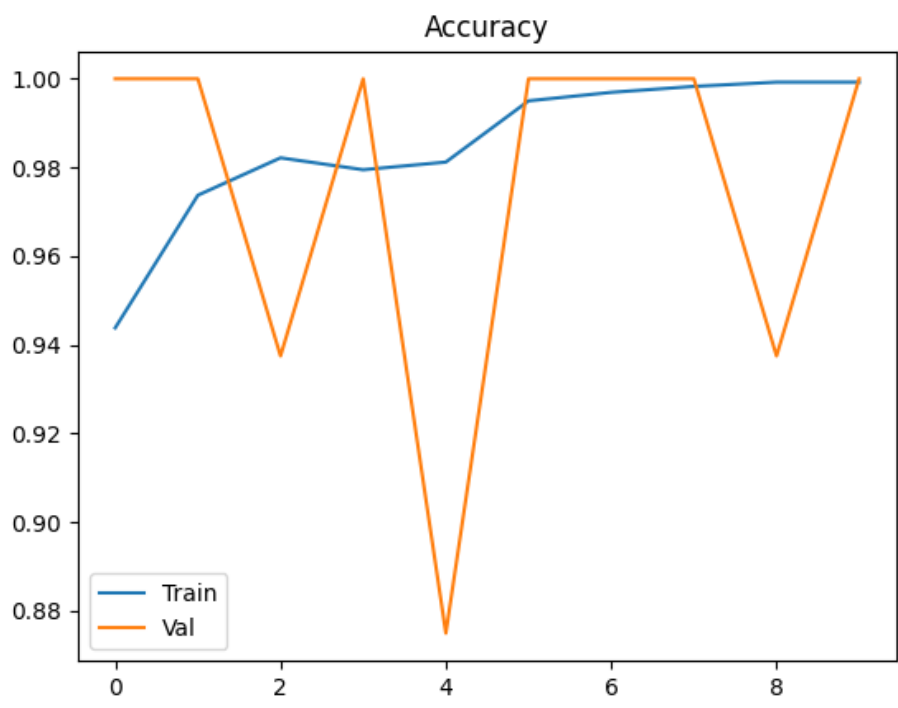
### Output:-

```
Epoch 1/10 ----- train Acc: 0.9438 val Acc: 1.0000
Epoch 2/10 ----- train Acc: 0.9737 val Acc: 1.0000
Epoch 3/10 ----- train Acc: 0.9822 val Acc: 0.9375
Epoch 4/10 ----- train Acc: 0.9795 val Acc: 1.0000
Epoch 5/10 ----- train Acc: 0.9812 val Acc: 0.8750
Epoch 6/10 ----- train Acc: 0.9950 val Acc: 1.0000
Epoch 7/10 ----- train Acc: 0.9969 val Acc: 1.0000
Epoch 8/10 ----- train Acc: 0.9983 val Acc: 1.0000
Epoch 9/10 ----- train Acc: 0.9992 val Acc: 0.9375
Epoch 10/10 ----- train Acc: 0.9992 val Acc: 1.0000
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 1.00   | 1.00     | 8       |
| 1            | 1.00      | 1.00   | 1.00     | 8       |
| accuracy     |           |        | 1.00     | 16      |
| macro avg    | 1.00      | 1.00   | 1.00     | 16      |
| weighted avg | 1.00      | 1.00   | 1.00     | 16      |

```
Model saved!
```

GRAPHS :-



## **MY COMMENTS :-**

**During this experiment, I learned how powerful transfer learning is compared to training a model from scratch. Initially, I faced issues with dataset handling and folder structure, but after correcting them, the model training worked properly.**

**I also understood the importance of preprocessing and how even small mistakes in dataset structure can cause errors. One interesting observation was that the model achieved very high validation accuracy quickly, which shows that the dataset was relatively simple or well-structured.**

**This project helped me gain practical understanding of PyTorch, deep learning workflows, and model evaluation techniques.**

**Overall, this experiment was very useful in understanding real-world applications of deep learning in the medical field.**